

Машина для подачи мячей

Мадина изобрела машину для подачи мячей, чтобы развлечь участников IOI. Внутренняя структура машины выглядит следующим образом:

- Машина состоит из N **узлов**, пронумерованных от 0 до $N - 1$. Узел $N - 1$ называется **корнем**.
- Для каждого узла u , где $0 \leq u < N - 1$ **родителем** узла u является узел $P[u]$, где $P[u] > u$, а узел u является **ребенком** узла $P[u]$. У корня нет родителя. Узел, не имеющий детей, называется **листом**.

Вам **не** известны ни число N , ни массив родителей $P[u]$. Вместо этого Мадина сообщает вам M , количество листьев и то, что индексы листьев — $0, 1, \dots, M - 1$. Ваша задача — определить внутреннюю структуру машины.

Каждый узел машины может содержать не более одного **шарика**, каждый шарик имеет **значение**, представляющее собой неотрицательное целое число. Будем говорить, что узел имеет значение x , если он содержит шарик со значением x . Узел считается **занятым**, если он содержит шарик; в противном случае он **свободен**. Изначально все узлы свободны.

Вы можете выполнять два вида операций:

- **insert** (U, X): попытаться вставить новый шарик со значением X в лист U .
 - Если лист U занят, операция возвращает `false` и не вставляет шарик в машину.
 - Если лист U свободен, шарик помещается в узел U . Затем шарик перемещается в родительский узел текущего узла до тех пор, пока родительский узел свободен. Перемещение останавливается, когда шарик достигает корня или узла, родительский узел которого занят. Операция возвращает `true`.
- **collect**(): если корень свободен (что означает, что в машине нет ни одного шарика), `collect` возвращает пустой массив. В противном случае рекурсивная функция `traverse`, описанная ниже, собирает значения всех шариков, которые в данный момент находятся в машине, в массив S . Изначально массив S пуст, и вызывается `traverse(N - 1)`.

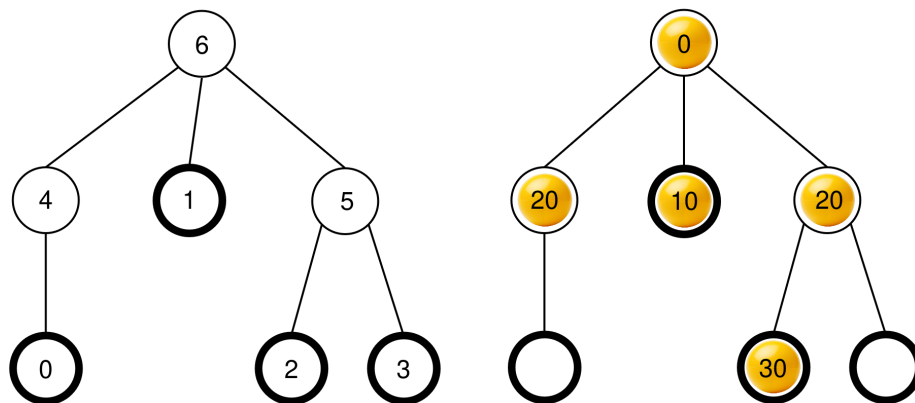
```

traverse(u) :
    добавить значение узла u в конец массива S
    создать массив c[u] — список занятых детей узла u
    отсортировать c[u] в неубывающем порядке, используя в качестве
        ключа значения шариков в узлах (если несколько узлов имеют одно
        и то же значение, то они могут в результате сортировки оказаться в
        любом порядке)
    для каждого узла v в массиве c[u] вызвать:
        traverse(v)

```

После завершения этой функции **все шарики удаляются** из машины, и `collect` возвращает массив S .

Например, рассмотрим машину с $N = 7$ узлами и массивом родительских узлов $P = [4, 6, 5, 5, 6, 6]$. На изображении слева показана пустая машина с индексами узлов, а на изображении справа — возможная конфигурация шариков после некоторой последовательности операций `insert` (эта последовательность приведена в разделе «Пример»).



При вызове функции `collect()` корневой узел (6) занят, поэтому переменная S инициализируется как $S = []$, после чего вызывается функция `traverse(6)`.

traverse(6): узел 6 имеет значение 0; при добавлении его к S получается $S = [0]$. Детями 6 являются 4, 1 и 5, все из которых заняты. Их значения, соответственно, равны 20, 10 и 20. Сортировка в неубывающем порядке даёт $c[6] = [1, 5, 4]$ (обратите внимание, что $c[6] = [1, 4, 5]$ также было бы допустимо, поскольку узлы 4 и 5 имеют одинаковое значение). Затем функция вызывает `traverse` для каждого узла в $c[6]$ в указанном порядке.

- **traverse(1):** узел 1 имеет значение 10; добавление его к S даёт $S = [0, 10]$. Узел 1 не имеет занятых детей, поэтому этот вызов заканчивается.
- **traverse(5):** узел 5 имеет значение 20; при добавлении его к S получается $S = [0, 10, 20]$. Узел 5 имеет одного занятого ребенка — узел 2, поэтому выполняется $c[5] = [2]$, и функция вызывает `traverse(2)`.

- `traverse(2)`: узел 2 имеет значение 30; при добавлении его к S получается $S = [0, 10, 20, 30]$. Узел 2 не имеет занятых детей, поэтому этот вызов завершается.
- Выполнение возвращается к вызову `traverse(5)`, который обработал всех детей в $c[5]$, поэтому завершается.
- `traverse(4)`: узел 4 имеет значение 20; присоединение его к S даёт $S = [0, 10, 20, 30, 20]$. Узел 4 не имеет занятых детей, поэтому этот вызов завершается.

Все вызовы завершены, и больше нет узлов для обработки. Наконец, все шарики извлекаются из машины, и функция `collect` возвращает массив $S = [0, 10, 20, 30, 20]$.

Ваша задача — определить количество узлов N и вывести массив родительских узлов $R = [R[0], R[1], \dots, R[N-2]]$, описывающий структуру машины, но с потенциально иной маркировкой узлов, которые не являются листьями и не являются корнем. Формально, возвращаемый массив R считается правильным, если каждому узлу u машины можно присвоить уникальную метку $L[u]$, где $0 \leq L[u] < N$, таким образом, чтобы:

- $L[u] = u$ для всех $0 \leq u < M$ и $u = N-1$, и
- $L[P[u]] = R[L[u]]$ для всех $0 \leq u < N-1$.

Не требуется, чтобы $R[u] > u$. Машина **не должна содержать шариков** при выводе решения.

Пусть K — количество выполненных операций `collect`, а B — максимальное значение, записанное на любом шарике при всех вызовах `insert`. Пусть $C = K + B$. **Значение C не должно превышать 1000**. Ваш балл в некоторых подзадачах зависит от значения C .

Детали реализации

Вам необходимо реализовать следующую функцию.

```
std::vector<int> find_structure(int M)
```

- M : количество листьев машины.
- Функция вызывается ровно один раз для каждого теста.
- Функция должна возвращать массив $R = [R[0], R[1], \dots, R[N-2]]$ длиной $N-1$: корректный массив родителей.

Для взаимодействия с машиной ваша функция может вызывать следующие две функции.

Первая функция:

```
bool insert(int U, int X)
```

- U : индекс листа, в который следует вставить шарик. Должно выполняться условие $0 \leq U < M$.
- X : целое число — значение шарика. Должно выполняться условие $0 \leq X \leq 1000$.

- Эта функция возвращает `true`, если шар был успешно вставлен, или `false`, если лист U уже был занят.
- Эта функция может быть вызвана не более 500 000 раз.

Вторая функция:

```
std::vector<int> collect()
```

- собирает значения всех вставленных шариков, как описано в условии задачи, удаляет все шарики из машины и возвращает массив собранных значений S .

Если в любой момент во время выполнения вашей программы значение C превысит 1000, ваше решение получит оценку `Output isn't correct: Too many resources used`.

Структура машины фиксируется до вызова функции `find_structure`. Грейдер является **детерминированным** в том смысле, что если запустить программу дважды и она выполнит одну и ту же последовательность операций, результаты вызовов `collect` будут одинаковыми.

Ограничения

- $2 \leq N \leq 1000$
- $1 \leq M \leq 200$
- $M < N$

Подзадачи

Подзадача	Баллы	Дополнительные ограничения
1	5	У корня ровно M детей.
2	10	$M \leq 3$
3	25	$N \leq 200, M \leq 45$
4	60	Дополнительных ограничений нет.

В подзадачах 3 и 4 ваши баллы зависят от значения C следующим образом.

Подзадача 3 (25 баллов)

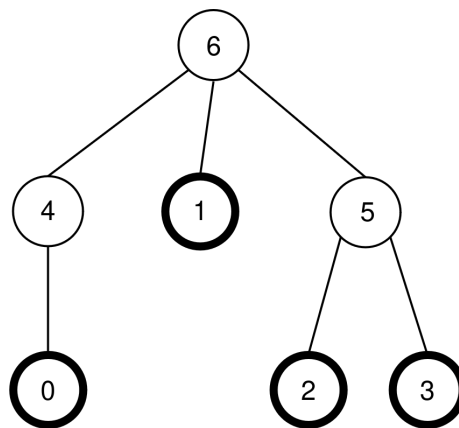
Ограничения	Баллы
$1000 < C$	0
$45 < C \leq 1000$	13
$C \leq 45$	25

Подзадача 4 (60 баллов)

Ограничения	Оценка
$1000 < C$	0
$200 < C \leq 1000$	7
$71 < C \leq 200$	$47 - \frac{C}{5}$
$44 < C \leq 71$	$104 - C$
$C \leq 44$	60

Пример

Рассмотрим ту же машину, что и в описании задачи, то есть $N = 7$, $M = 4$ и $P = [4, 6, 5, 5, 6, 6]$. Машина снова показана на следующем рисунке:



Пусть выполняется следующий вызов:

```
find_structure(4)
```

Функция может выполнить следующую последовательность вызовов:

1. **insert(0, 0)**: лист 0 свободен, поэтому шарик со значением 0 помещается в лист 0. Узел $P[0] = 4$ свободен, поэтому шарик перемещается в узел 4. Узел $P[4] = 6$ свободен, поэтому шарик перемещается в узел 6. Поскольку узел 6 является корнем, перемещение останавливается. Вызов возвращает `true`.
2. **insert(3, 20)**: лист 3 свободен, поэтому шарик со значением 20 помещается в лист 3. Узел $P[3] = 5$ свободен, поэтому шарик перемещается в узел 5. Поскольку узел $P[5] = 6$ занят, перемещение останавливается. Вызов возвращает `true`.
3. **insert(1, 10)**: лист 1 свободен, поэтому шарик со значением 10 помещается в лист 1. Поскольку узел $P[1] = 6$ занят, шарик остаётся в узле 1. Вызов возвращает `true`.
4. **insert(2, 30)**: лист 2 свободен, поэтому шарик со значением 30 помещается в лист 2. Поскольку узел $P[2] = 5$ занят, шарик остаётся в узле 2. Вызов возвращает `true`.

5. `insert(1, 25)`: лист 1 занят, поэтому шарик не помещается в лист 1. Вызов возвращает `false`.
6. `insert(0, 20)`: лист 0 свободен, поэтому шарик со значением 20 помещается в лист 0. Узел $P[0] = 4$ свободен, поэтому шарик перемещается в узел 4. Поскольку узел $P[4] = 6$ занят, перемещение останавливается. Вызов возвращает `true`.

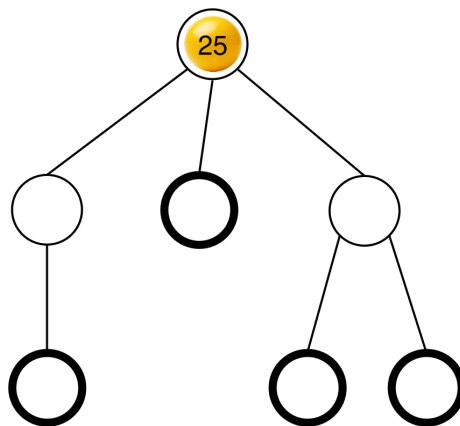
Результирующая конфигурация шариков уже была показана в описании задачи.

Далее функция может вызвать:

```
collect()
```

Выполнение этого вызова уже было объяснено в описании задачи, поэтому этот вызов возвращает $S = [0, 10, 20, 30, 20]$. После этого все шарики удаляются из машины и все узлы становятся пустыми.

Далее функция может вызвать `insert(2, 25)`. Лист 2 свободен, поэтому шарик со значением 25 помещается в лист 0. Затем этот шарик перемещается в узел 5, а затем в узел 6, где останавливается. Вызов возвращает `true`. Результирующая конфигурация шаров показана на следующем рисунке.



Наконец, функция может вызвать `collect()`, которая возвращает $S = [25]$ и очищает машину.

Существует два корректных массива родителей, которые может вернуть функция `find_structure`: $R = P = [4, 6, 5, 5, 6, 6]$ (соответствующий маркировке $L = [0, 1, 2, 3, 4, 5, 6]$) и $R = [5, 6, 4, 4, 6, 6]$ (соответствующий маркировке $L = [0, 1, 2, 3, 5, 4, 6]$). В данном примере $K = 2$ и $B = 30$, поэтому $C = 32$.

Пример грейдера

Формат входных данных:

```
N M
P[0] P[1] P[2] ... P[N-2]
```

Формат выходных данных:

```
K B C
Q
R[0] R[1] R[2] ... R[Q-1]
```

Здесь Q — это длина массива R , возвращаемого функцией `find_structure`.